SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192421166
Name	DHILIBAN M

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

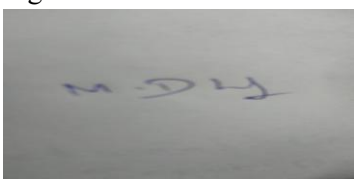
Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**
Question Count: 10

Code of Conduct

I DHILIBAN M/192421166 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Mock Interview

Answer the following interview-oriented questions clearly and concisely. Support your responses with architecture-level reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?
8. What are the key failure points in a Hadoop cluster and how would you handle them?
9. How would you design a secure data ingestion pipeline for a large organization?
10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mock Interview Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Knowledge	25%	Shows deep and accurate technical understanding	Good technical understanding	Basic understanding	Weak understanding
Problem Solving	20%	Responds with strong architecture-level reasoning	Reasonable reasoning	Basic reasoning	Weak reasoning
Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication
Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

1. HDFS vs. Traditional File Systems

Traditional file systems (like NTFS or ext4) are designed for single-machine storage — they assume one disk, one point of failure, and vertical scaling (bigger disk/server) when capacity runs out.

HDFS (Hadoop Distributed File System) is designed for horizontal scaling across commodity hardware. Key architectural differences:

- **Distribution**: HDFS splits files into large blocks (default 128MB) and spreads them across many nodes in a cluster, rather than storing a file contiguously on one disk.
- **Fault tolerance by design**: Traditional file systems rely on RAID or backups for redundancy. HDFS replicates each block (default 3x) across different nodes/racks natively, so hardware failure is an expected condition, not an exception.
- **Write-once, read-many model**: HDFS is optimized for large sequential reads/writes and batch processing, not frequent random writes or low-latency access like a traditional FS.
- **Master-slave architecture**: A NameNode holds metadata (the "map" of blocks to locations); DataNodes store actual blocks. Traditional file systems have no such separation — metadata and data live together on the same volume.

In short: traditional file systems optimize for single-machine reliability and general-purpose access; HDFS optimizes for massive scale, fault tolerance, and throughput on batch workloads.

2. Why Replication Matters in Distributed Storage

At scale, hardware failure isn't a rare event — with thousands of commodity disks, something is failing at any given time. Replication addresses three things:

- **Fault tolerance**: If a DataNode fails, replicas on other nodes keep data available with zero downtime.
- **Data locality for compute**: Since compute is often scheduled where data resides (bring compute to data, not data to compute), having multiple replicas increases the chance a task can run on a node with local data, reducing network I/O.
- **Read throughput**: Multiple replicas allow multiple clients to read the same data in parallel from different nodes, avoiding a single-node bottleneck.

The tradeoff is storage overhead — 3x replication means 3x raw storage cost, which is why systems like HDFS Erasure Coding (introduced in Hadoop 3.x) exist as a more storage-efficient alternative for colder data, trading some fault-tolerance speed for reduced storage overhead (roughly 1.5x instead of 3x).

3. How MapReduce Divides Work

MapReduce splits a job into two distinct phases connected by a shuffle-and-sort step:

- **Map phase**: Input data (from HDFS) is split into chunks, and a Mapper function runs in parallel across nodes, transforming raw input into intermediate key-value pairs. Each mapper works independently on its own data split — this is where most of the parallelism comes from.
- **Shuffle & Sort** (between phases): The framework redistributes intermediate data so that all values for the same key end up on the same reducer, sorting keys along the way. This is often the most network-intensive part of the job.

- **Reduce phase**: Reducer functions take all values associated with a given key and aggregate/summarize them into final output, which is written back to HDFS.

Architecturally, the key idea is **data locality**: mappers are scheduled to run on (or near) the node holding the relevant data block, minimizing data movement. Only the shuffle phase forces significant network transfer.

4. YARN's Role in Hadoop

YARN (Yet Another Resource Negotiator) decouples resource management from processing logic — a major architectural shift from Hadoop 1.x, where MapReduce handled both.

- **ResourceManager**: A global component that tracks cluster resources (CPU, memory) and allocates them to applications.
- **NodeManager**: Runs on each worker node, monitors resource usage, and reports back to the ResourceManager.
- **ApplicationMaster**: A per-job component that negotiates resources and coordinates execution of that specific application's tasks.

This separation means YARN isn't tied to MapReduce alone — it can schedule resources for Spark, Tez, Hive, or any YARN-compatible engine. This turned Hadoop from a single-purpose MapReduce system into a general-purpose cluster resource management platform, which is why the ecosystem diversified so much after Hadoop 2.x.

5. NAS vs. SAN

Both are network-based storage architectures, but they operate at different layers:

Aspect	**NAS (Network Attached Storage)**	**SAN (Storage Area Network)**
Access level	File-level (via NFS/SMB protocols)	Block-level (via Fibre Channel/iSCSI)
Appears to OS as	A network file share	A local disk/volume
Typical use case	Shared file access across users/departments	High-performance, low-latency storage for databases, VMs
Network	Uses standard Ethernet/IP	Often a dedicated high-speed network
Management complexity	Simpler to set up and manage	More complex, requires specialized SAN admin skills

In an enterprise context: NAS is a good fit for shared document repositories or general file storage where ease of access matters more than raw performance. SAN is preferred for transactional databases, virtualization clusters, or any workload needing high IOPS and low latency, because block-level access lets the OS/application manage the filesystem directly rather than going through a network file protocol.

6. Purpose of ETL in a Big Data Pipeline

ETL (Extract, Transform, Load) exists to convert raw, heterogeneous data into a clean, consistent, analysis-ready form:

- **Extract**: Pull data from disparate sources — databases, APIs, logs, flat files, streaming systems.

- **Transform**: Clean, validate, deduplicate, standardize formats/units, apply business logic, join/enrich with reference data.
- **Load**: Write the processed data into a target system — a data warehouse, data lake, or serving layer — optimized for downstream querying or analytics.

Architecturally, ETL exists because raw source data is rarely fit for direct analytical use — it has inconsistent schemas, missing values, duplicate records, and format mismatches across systems. In modern big data architectures, this is often inverted to **ELT** (load raw data first, transform on-demand within the warehouse/lake using its compute power) — a pattern common with tools like Snowflake or BigQuery, since storage is cheap and compute can be applied flexibly at query time rather than committing to a fixed schema upfront.

7. How Apache NiFi Helps with Data Integration

NiFi is a visual, flow-based tool for automating data movement between systems, and its architecture solves a few specific integration problems:

- **Flow-based programming model**: Data is represented as "FlowFiles" moving through a directed graph of processors (each processor does one job — route, transform, filter, enrich). This makes complex pipelines visually inspectable and easier to modify than hand-coded scripts.
- **Data provenance**: NiFi tracks the full lineage of every piece of data as it moves through the flow — critical for debugging and compliance/audit requirements.
- **Back-pressure and flow control**: NiFi can throttle data flow between processors, preventing downstream systems from being overwhelmed — useful when source and destination systems have very different throughput capacities.
- **Guaranteed delivery**: Uses a write-ahead log design so data isn't lost even if a node crashes mid-flow.

In practice, NiFi is often used at the ingestion layer of a big data pipeline — bringing data in from IoT devices, APIs, or file drops, doing lightweight routing/transformation, and handing it off to Kafka, HDFS, or a database for further processing.

8. Key Failure Points in a Hadoop Cluster

- **NameNode failure**: Historically the single point of failure in Hadoop 1.x, since it holds all filesystem metadata. Mitigated in Hadoop 2.x+ via **NameNode HA** (active/standby pair with automatic failover via ZooKeeper) and **Secondary/Checkpoint NameNode** to periodically merge edit logs.
- **DataNode failure**: Handled natively — the NameNode detects missed heartbeats, marks the node dead, and triggers re-replication of under-replicated blocks from surviving nodes.
- **Network partitioning/rack failure**: HDFS's rack-awareness places replicas across racks specifically so a single rack or switch failure doesn't cause data loss.
- **ResourceManager failure**: A single point of failure for job scheduling; mitigated with **ResourceManager HA** (active/standby, also via ZooKeeper).
- **Job/task failure**: Individual mapper/reducer task failures are handled by retrying the task on another node (configurable retry count) before failing the whole job.
- **Disk failure on a node**: Handled at the HDFS replication layer, though monitoring disk health proactively (e.g., via Cloudera Manager/Ambari) helps avoid cascading issues.

The general design philosophy: assume failure is normal, build automatic detection and recovery in at every layer, and eliminate single points of failure through HA pairs coordinated by ZooKeeper.

9. Designing a Secure Data Ingestion Pipeline

For a large organization, I'd think about this in layers:

- **Authentication**: Kerberos for cluster-wide authentication (standard in Hadoop security), integrated with the org's existing identity provider (LDAP/Active Directory) so credentials aren't managed separately.
- **Authorization**: Fine-grained access control via Apache Ranger or Sentry — role-based policies at the database, table, or even column level, not just OS-level file permissions.
- **Encryption in transit**: TLS/SSL for all data movement between ingestion tools, brokers (e.g., Kafka), and storage.
- **Encryption at rest**: HDFS Transparent Data Encryption (TDE) with encryption zones, or storage-layer encryption if on cloud object storage.
- **Ingestion layer controls**: Use a tool like NiFi or Kafka Connect with schema validation at the entry point, so malformed or malicious payloads are rejected before they reach the lake.
- **Network isolation**: Ingestion endpoints in a DMZ/segmented network, with a controlled, audited path into the internal cluster.
- **Auditing and lineage**: Full audit logging (Ranger audit, NiFi provenance) so every access and transformation is traceable — critical for compliance frameworks like GDPR or HIPAA.
- **Data classification**: Tag sensitive fields (PII, financial data) at ingestion time so downstream masking/tokenization policies can apply automatically.

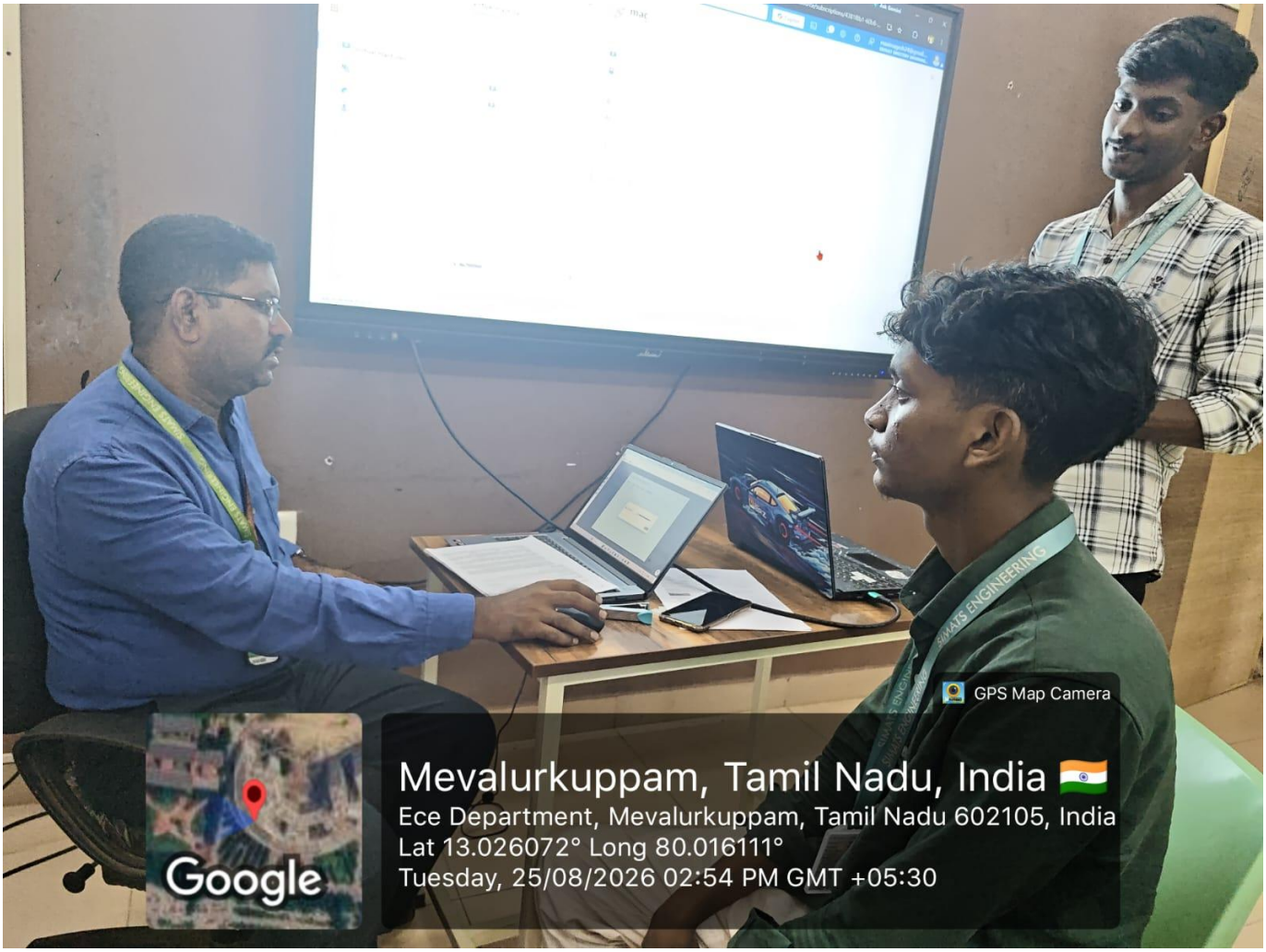
The core principle is defense in depth — no single control (like a firewall) is sufficient; security has to be enforced at network, authentication, authorization, and data levels simultaneously.

10. Explaining a Scalable Hadoop System to a Non-Technical Manager

"Think of it like a library that keeps growing faster than any single building could hold. Instead of building one giant warehouse, we spread the books across many smaller buildings, and we keep copies of each book in at least three different buildings — so if one building has a problem, we never lose anything.

When we need to answer a question — like 'how many customers bought X last month' — instead of sending one person to search the entire spread-out library, we send a small team to each building at the same time, have them each search their own section, and then combine their answers at the end. That's what makes it fast even as the library keeps growing: we're not limited by how fast one person can search: we just add more buildings and more search teams as we grow.

The business benefit is cost and scalability — instead of buying one very expensive, very powerful computer, we use many affordable ones working together, and we can add more as data grows without redesigning the whole system."


GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
Ece Department, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.026072° Long 80.016111°
Tuesday, 25/08/2026 02:54 PM GMT +05:30
Google